Búsqueda de una configuración de prompt bajo presupuesto de consultas: formalización, dificultad computacional y solución base del Proyecto Oráculo

Juan Manuel Carmona Villegas

Programa de Ingeniería de Sistemas y Computación, Universidad de Caldas

Análisis y Diseño de Algoritmos · Grupo G01 · Primera entrega (semanas 1–4)

Docente: Luz Enith Guerrero Mendieta

6 de noviembre de 2026

Repositorio: github.com/Manuel29296/ADA-2026-2-GP01

Tabla de contenido

1. Introducción y planteamiento del problema
2. Formalización · Parte 1: configuración óptima con oráculo exacto
    2.1 Entrada
    2.2 Espacio de soluciones
    2.3 Tamaño del problema
    2.4 Salida
    2.5 Función objetivo
    2.6 Restricciones de validez
    2.7 Versión de decisión
    2.8 Costo de la búsqueda exhaustiva
3. Formalización · Parte 2: búsqueda con consultas limitadas y ruidosas
    3.1 Qué se quiere maximizar realmente
    3.2 Costo de una consulta y presupuesto
    3.3 El ruido de medir con pocas instancias
    3.4 Complejidad de muestreo
4. Supuestos y alcance
5. NP-dureza de la parte 1 por reducción desde 3-SAT
    5.1 Modelo de oráculo exacto usado en la prueba
    5.2 El problema de origen: 3-SAT
    5.3 PROMPT-DEC pertenece a NP
    5.4 La transformación
    5.5 Equivalencia de las respuestas (si y solo si)
    5.6 Ejemplo
    5.7 Conclusión de la reducción
6. Proceso de optimización
    6.1 Justificación del método
    6.2 Fase 1: cribado
    6.3 Uso de las trazas para podar
    6.4 Fase 2: interacción entre ranuras
    6.5 Fase 3: ascenso sin poda
    6.6 Optimalidad del resultado

7. Resultados, validación y reproducibilidad

7.1 Configuración final

7.2 Análisis de fallos: línea base frente a configuración final

7.3 Validación

7.4 Limitaciones

7.5 Reproducibilidad

Referencias

1. Introducción y planteamiento del problema

El proyecto busca la combinación de instrucciones de prompt que hace que un modelo de lenguaje pequeño cumpla el mayor número posible de reglas de formato verificables, consultando lo menos posible al modelo.

El modelo usado es Qwen3-1.7B (alias `qwen17b`), ejecutado en una GPU T4 de Google Colab. Cada solicitud al modelo viene acompañada de reglas como «envuelve toda la respuesta entre comillas» o «no uses puntos». Una solicitud se considera acertada solo si la respuesta cumple todas sus reglas (criterio todo-o-nada).

El modelo no se reentrena. Lo único que se decide es qué texto instruccional se agrega alrededor de cada solicitud. El catálogo del curso ofrece cinco ranuras, cada una con ocho opciones (Tabla 1).

Tabla 1

Ranuras del prompt y su posición

Ranura	**Posición en el prompt**	**Qué orienta**
rol	antes de la solicitud	el papel que asume el modelo
estrategia	antes de la solicitud	cómo razonar o construir la respuesta
formato	antes de la solicitud	la estructura de la respuesta
estilo	antes de la solicitud	la redacción y los caracteres
cierre	después de la solicitud	lo último que lee el modelo antes de responder

Una configuración es la elección de un índice entre 0 y 7 por ranura, más una temperatura de generación. Con temperatura fija en 0.0 existen $8^5 = 32\,768$ configuraciones.

La calidad de una configuración se mide con un oráculo provisto por el curso, que funciona como caja negra: recibe una configuración y un lote de instancias y devuelve la precisión (fracción de instancias acertadas) y las trazas de fallo (la primera regla incumplida en cada instancia fallida). La métrica a maximizar es esa precisión sobre las 100 instancias de la partición de búsqueda.

Evaluar una configuración sobre las 100 instancias tomó 3,2 minutos medidos en la T4. Probar las 32 768 configuraciones requeriría unos 73 días de GPU, por lo que el problema exige un algoritmo de búsqueda que encuentre una buena configuración con pocas consultas.

Sin ninguna instrucción adicional, el modelo acierta el 16 % de las instancias de búsqueda. Esa es la línea base contra la que se compara toda configuración.

Nota sobre el enunciado general. El documento general del curso describe una versión preliminar del problema: |C| = 3840, cuatro opciones por categoría, las categorías Verificación y Recordatorio, 150 instancias y una reducción desde Cobertura Máxima. Este informe sigue la versión vigente para el Grupo 1: el catálogo del repositorio del curso (cinco ranuras de ocho opciones), las 100 instancias de búsqueda y 87 de validación, y la reducción desde 3-SAT que exige la guía de primera entrega.

2. Formalización · Parte 1: configuración óptima con oráculo exacto

En esta primera parte se supone que el oráculo responde sin error y que puede consultarse para cualquier configuración e instancia. Bajo ese supuesto, el problema es puramente combinatorio: elegir una opción por ranura que maximice el número de instancias acertadas. Esta es la parte cuya NP-dureza se demuestra en la sección 5.

2.1 Entrada

1. Un conjunto de k ranuras, numeradas $s = 1, \ldots, k$. En el proyecto, $k = 5$ (rol, estrategia, formato, estilo, cierre).
2. Para cada ranura s, un conjunto de opciones $O_s = \{0, 1, \ldots, m-1\}$. En el proyecto, $m = 8$.
3. Un conjunto finito de temperaturas T. En el proyecto, $T = \{0.0\}$; el curso permite $T = \{0.0, 0.3, 0.7\}$.
4. Un conjunto de n instancias $I = \{i_1, \ldots, i_n\}$. En la partición de búsqueda, $n = 100$. Cada instancia i trae una solicitud y un conjunto de restricciones

verificables $\rho(i) = \{\varphi_{i,1}, \ldots, \varphi_{i,r_i}\}$, con 2,5 restricciones por instancia en promedio.

5. Un oráculo $\mathrm{ok}(c, i) \in \{0, 1\}$. Vale 1 si la respuesta $y(c, i)$ que genera el modelo con la configuración c para la instancia i cumple todas las restricciones de $\rho(i)$, y 0 si incumple al menos una:

$$\mathrm{ok}(c, i) \;=\; \prod_{\varphi \in \rho(i)} 1\big[\, y(c, i) \text{ cumple } \varphi \,\big]$$

El producto expresa el criterio todo-o-nada: basta un factor igual a 0 para que la instancia valga 0.

2.2 Espacio de soluciones

Una configuración es una tupla con una opción por ranura y una temperatura. El espacio de todas las configuraciones es el producto cartesiano:

$$C \;=\; O_1 \times O_2 \times \cdots \times O_k \times T, \qquad |C| \;=\; m^{k} \cdot |T|$$

Con $k = 5$, $m = 8$ y $T = \{0.0\}$, $|C| = 8^5 = 32\,768$. Con tres temperaturas, $|C| = 98\,304$.

2.3 Tamaño del problema

El tamaño no se reduce a una sola magnitud. Intervienen las de la Tabla 2.

Tabla 2

Magnitudes que definen el tamaño del problema

Símbolo	Significado	Valor en el proyecto	Qué hace crecer
k	número de ranuras	5	el número de configuraciones, de forma exponencial
m	opciones por ranura	8	el número de configuraciones, como base de la potencia
$\lvert T \rvert$	temperaturas posibles	1	el número de configuraciones, linealmente
n	instancias evaluadas	100 (búsqueda)	el costo de evaluar una configuración
r	restricciones por instancia	2,5 en promedio	el costo de verificar una respuesta
L	longitud máxima de respuesta	384 tokens	el costo de generar una respuesta

El crecimiento dominante es exponencial en k: cada ranura adicional multiplica por m el espacio. Con $m = 8$, pasar de 5 a 6 ranuras lleva el espacio de 32 768 a 262 144 configuraciones.

2.4 Salida

Una configuración $c^* = (o_1, o_2, o_3, o_4, o_5, t) \in C$, que se entrega en `entrega.json` como los índices de rol, estrategia, formato, estilo y cierre, más la temperatura.

2.5 Función objetivo

Se maximiza la precisión sobre el conjunto de instancias, es decir, la fracción de instancias acertadas:

$$\mathrm{Acc}_I(c) \;=\; \frac{1}{n} \sum_{i \in I} \mathrm{ok}(c, i), \qquad c^* \in \arg\max_{c \in C} \mathrm{Acc}_I(c)$$

2.6 Restricciones de validez

Una solución es válida si y solo si pertenece a C: exactamente un índice $o_s \in O_s$ por cada ranura y una temperatura $t \in T$. No se admiten ranuras vacías ni índices fuera de 0–7,

porque `entrega.json` exige los cinco índices.

2.7 Versión de decisión

Para el análisis de complejidad se usa la versión de decisión del problema, llamada PROMPT-DEC:

PROMPT-DEC. Dados las ranuras con sus opciones, las instancias I, el oráculo ok y un umbral entero θ con $0 \le \theta \le n$, ¿existe una configuración $c \in C$ con $\sum_i \mathrm{ok}(c, i) \ge \theta$?

Si se pudiera resolver el problema de optimización, se resolvería también PROMPT-DEC, comparando la precisión de c^* con θ. Por eso, si PROMPT-DEC es NP-difícil, el problema de optimización también lo es.

2.8 Costo de la búsqueda exhaustiva

Aun con oráculo exacto, la fuerza bruta evalúa $\mathrm{ok}(c, i)$ para todo $c \in C$ e $i \in I$:

$$T_{\text{exh}}(k, m, |T|, n) \;=\; |C| \cdot n \;=\; m^k \, |T| \, n \;\in\; O\big(m^k \, |T| \, n\big)$$

En el proyecto son 32 768 × 100 = 3 276 800 consultas al modelo. La deducción formal paso a paso de $T(n)$ para el algoritmo exacto corresponde a las semanas 5–6.

3. Formalización · Parte 2: búsqueda con consultas limitadas y ruidosas

En la práctica, el oráculo no es gratuito ni exacto respecto de lo que se califica. Cada consulta cuesta tiempo de GPU, y la precisión medida sobre 100 instancias es solo una estimación de la precisión sobre el conjunto de prueba privado de la docente. La demostración de NP-dureza de la sección 5 no se traslada a esta parte. Aquí el análisis es de costo y de complejidad de muestreo.

3.1 Qué se quiere maximizar realmente

Las instancias de búsqueda, las de validación y las de prueba privada provienen de una misma fuente. Se modelan como muestras de una distribución $\mathcal{D}$ de instancias. La precisión

verdadera de una configuración es su probabilidad de acierto sobre una instancia nueva:

$$\mathrm{Acc}_{\mathcal{D}}(c) \;=\; \Pr_{i \sim \mathcal{D}}\left[\mathrm{ok}(c, i) = 1\right]$$

Esa cantidad no se puede observar. Lo que se observa es la precisión empírica sobre una muestra S de instancias:

$$\widehat{\mathrm{Acc}}_S(c) \;=\; \frac{1}{|S|} \sum_{i \in S} \mathrm{ok}(c, i)$$

3.2 Costo de una consulta y presupuesto

Una consulta es una generación del modelo para una pareja (configuración, instancia). Medir una configuración sobre las 100 instancias de búsqueda cuesta 100 consultas, que tomaron 3,2 minutos en la T4 (unos 1,9 segundos por consulta). Las consultas repetidas no cuestan: el oráculo guarda en caché cada respuesta generada.

Un algoritmo de búsqueda decide cuántas consultas m_c dedica a cada configuración c, y al final propone una configuración $\hat{c}$. El problema con presupuesto B es:

$$\max_{\text{algoritmo}} \; \mathrm{Acc}_{\mathcal{D}}(\hat{c}) \qquad \text{sujeto a} \qquad \sum_{c \in C} m_c \;\le\; B$$

El oráculo del curso no impone un valor de B; el límite real es el tiempo de GPU disponible en Colab. La búsqueda exhaustiva necesitaría $B = 3\,276\,800$ consultas (unos 73 días), lo que obliga a usar B mucho menores.

3.3 El ruido de medir con pocas instancias

Con temperatura 0.0 el modelo es determinista: la misma configuración y la misma instancia producen siempre la misma respuesta. El ruido no viene de la generación, sino de qué instancias se usan para medir. Cada instancia es un ensayo de acierto o fallo, así que el error típico de la precisión medida es:

$$\mathrm{EE}\big(\widehat{\mathrm{Acc}}_S\big) \;\approx\; \sqrt{\frac{p\,(1-p)}{|S|}}$$

Con $p \approx 0{,}18$ y $|S| = 100$, el error típico es de unos 3,8 puntos porcentuales. Por eso, dos configuraciones que difieren en 1 o 2 puntos sobre 100 instancias no son distinguibles con seguridad.

3.4 Complejidad de muestreo

La desigualdad de Hoeffding (1963) acota la probabilidad de que la precisión medida se aleje más de ε de la verdadera:

$$\Pr\left[\left|\widehat{\mathrm{Acc}}_S(c) - \mathrm{Acc}_{\mathcal{D}}(c)\right| \geq \varepsilon\right] \;\leq\; 2\,e^{-2|S|\varepsilon^2}$$

Despejando, para que el error sea menor que ε con probabilidad al menos $1-\delta$ basta con:

$$|S| \;\geq\; \frac{\ln(2/\delta)}{2\,\varepsilon^2}$$

Si se comparan K configuraciones a la vez y se quiere que todas queden bien medidas simultáneamente, la cota de la unión reemplaza δ por δ/K:

$$|S| \;\geq\; \frac{\ln(2K/\delta)}{2\,\varepsilon^2}$$

Tabla 3

Instancias necesarias según la cota de Hoeffding (δ = 0,05)

Situación	ε	Instancias necesarias
Una configuración	0,10	185
Una configuración	0,05	738
40 configuraciones a la vez	0,10	369
Con las 100 de búsqueda (una configuración)	0,136	100

Estas cotas (Tabla 3) son conservadoras, porque Hoeffding no usa la forma de la distribución. Aun así, muestran que 100 instancias no bastan para garantizar diferencias pequeñas. Dos decisiones del diseño mitigan el problema:

- Todas las configuraciones se miden sobre las mismas 100 instancias, con la misma semilla. Así, la comparación es pareada: las instancias fáciles o difíciles afectan por igual a todas.
- La configuración final se comprueba con `oraculo.validar` sobre 87 instancias que no se usaron para elegirla, lo que da una segunda estimación independiente.

4. Supuestos y alcance

1. **Oráculo como caja negra.** El archivo `oraculo.py` se usa solo a través de `evaluar` y `validar`. No se lee su contenido para elegir la configuración.
2. **Modelo fijo.** Todas las mediciones usan Qwen3-1.7B (`qwen17b`), el modelo por defecto del curso, en una GPU T4 de Colab. Se supone que la evaluación de la docente usa el mismo modelo.
3. **Temperatura 0.0.** La generación es determinista, así que cada pareja (configuración, instancia) tiene una única respuesta y el caché es válido. Las temperaturas 0.3 y 0.7 quedan fuera del alcance de esta entrega.
4. **Trazas parciales.** Cada traza informa solo la primera restricción incumplida, no todas. El análisis de fallos trabaja con esa información parcial.
5. **Particiones del curso.** Se usan las 100 instancias de búsqueda y las 87 de validación definidas por `dividir`. Las 263 instancias marcadas como descartadas no se usan.
6. **Datos privados fuera de alcance.** El conjunto de prueba con el que la docente calificará no está disponible para el grupo. Puede incluir familias de restricciones que no aparecen en los datos visibles.

7. **Resultado no garantizado.** La búsqueda entrega la mejor configuración encontrada con el presupuesto usado, no un óptimo global demostrado.
8. **Costo medido, no teórico.** Los tiempos reportados (3,2 minutos por evaluación de 100 instancias) dependen de la GPU asignada por Colab y pueden variar entre sesiones.

5. NP-dureza de la parte 1 por reducción desde 3-SAT

Se demuestra que PROMPT-DEC es NP-completo y, por tanto, que encontrar la configuración óptima con oráculo exacto es NP-difícil. La prueba transforma cualquier fórmula de 3-SAT en una instancia de PROMPT-DEC en tiempo polinomial ($\text{3-SAT} \leq_p \text{PROMPT-DEC}$).

5.1 Modelo de oráculo exacto usado en la prueba

Para que la reducción sea precisa, cada instancia i se describe por una lista de parejas favorables $F(i) = \{(s, o), \dots\}$. Cada pareja (s, o) indica que elegir la opción o en la ranura s repara la regla de la instancia. El oráculo exacto vale:

$$\text{ok}(c, i) = 1 \iff \exists\,(s, o) \in F(i) \;:\; c_s = o$$

Este modelo es un caso particular del problema real. Una regla como «termina la respuesta con comillas» puede quedar reparada por una opción del cierre o por una del estilo. Si el caso particular ya es difícil, el caso general también lo es. Con esta descripción, $\text{ok}(c, i)$ se calcula en tiempo $O(|F(i)|)$.

5.2 El problema de origen: 3-SAT

3-SAT. Dada una fórmula booleana ϕ en forma normal conjuntiva sobre las variables $x_1, \dots, x_N$, con M cláusulas de exactamente tres literales cada una, ¿existe una asignación de verdadero o falso a las variables que haga verdaderas todas las cláusulas?

3-SAT es NP-completo (Cook, 1971; Garey & Johnson, 1979; Karp, 1972).

5.3 PROMPT-DEC pertenece a NP

El certificado es una configuración c. Para verificarlo se calcula $\mathrm{ok}(c, i)$ para las n instancias y se compara la suma con θ. Con el oráculo exacto del modelo 5.1, esto toma tiempo $O\big(\sum_i |F(i)|\big)$, polinomial en el tamaño de la entrada.

5.4 La transformación

Dada ϕ, se construye la instancia de PROMPT-DEC según la correspondencia de la Tabla 4.

Tabla 4

Correspondencia entre 3-SAT y PROMPT-DEC

3-SAT	**PROMPT-DEC**
Variable x_j ($j = 1, \ldots, N$)	Ranura $s = j$, con opciones $O_j = \{0, 1\}$ (0 = falso, 1 = verdadero)
Asignación σ	Configuración c, con $c_j = 1$ si y solo si $\sigma(x_j)$ = verdadero
Cláusula C_q ($q = 1, \ldots, M$)	Instancia i_q
Literal x_j en C_q	Pareja favorable $(j, 1)$ en $F(i_q)$
Literal $\neg x_j$ en C_q	Pareja favorable $(j, 0)$ en $F(i_q)$
¿Se satisfacen todas las cláusulas?	¿Existe c con $\sum_q \mathrm{ok}(c, i_q) \geq \theta$, con $\theta = M$?

La temperatura se fija en $T = \{0.0\}$. La construcción escribe N ranuras, M instancias y $3M$ parejas favorables, así que toma tiempo $O(N + M)$, polinomial en el tamaño de ϕ.

La reducción usa $m = 2$ opciones por ranura. Para $m = 8$, como en el proyecto, basta declarar que las opciones 1 a 7 de la ranura j equivalen a «x_j verdadero» y la opción 0 a «x_j falso», poniendo en $F(i_q)$ las siete parejas correspondientes. La equivalencia de abajo no cambia.

5.5 Equivalencia de las respuestas (si y solo si)

(⇒) Si ϕ es satisfacible, la instancia de PROMPT-DEC responde sí. Sea σ una asignación que satisface ϕ. Se define $c_j = 1$ si $\sigma(x_j)$ es verdadero y $c_j = 0$ si es falso. Cada cláusula C_q tiene al menos un literal verdadero bajo σ. Si ese literal es x_j, entonces $c_j = 1$ y $(j, 1) \in F(i_q)$. Si es $\neg x_j$, entonces $c_j = 0$ y $(j, 0) \in F(i_q)$. En ambos casos $\mathrm{ok}(c, i_q) = 1$. Como esto vale para las M instancias, $\sum_q \mathrm{ok}(c, i_q) = M \geq \theta$.

(⇐) Si la instancia de PROMPT-DEC responde sí, ϕ es satisfacible. Sea c una configuración con $\sum_q \mathrm{ok}(c, i_q) \geq M$. Como hay exactamente M instancias y cada ok vale 0 o 1, todas valen 1. Se define $\sigma(x_j)$ = verdadero si $c_j = 1$ y falso si $c_j = 0$. Para cada cláusula C_q, $\mathrm{ok}(c, i_q) = 1$ implica que existe $(j, o) \in F(i_q)$ con $c_j = o$. Si $o = 1$, el literal x_j está en C_q y es verdadero bajo σ. Si $o = 0$, el literal $\neg x_j$ está en C_q y es verdadero bajo σ. Entonces toda cláusula se satisface y ϕ es satisfacible.

5.6 Ejemplo

Sea $\phi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$. La reducción produce tres ranuras con opciones $\{0, 1\}$, dos instancias con $F(i_1) = \{(1, 1), (2, 0), (3, 1)\}$ y $F(i_2) = \{(1, 0), (2, 1), (3, 1)\}$, y $\theta = 2$. La Tabla 5 evalúa tres configuraciones.

Tabla 5

Ejemplo de la reducción con dos cláusulas

Configuración (c_1, c_2, c_3)	$\mathrm{ok}(c, i_1)$	$\mathrm{ok}(c, i_2)$	**Suma**	**¿$\geq \theta$?**
(0, 0, 0)	1 (por $c_2 = 0$)	1 (por $c_1 = 0$)	2	sí
(1, 0, 0)	1	0	1	no
(0, 0, 1)	1	1	2	sí

La configuración (0, 0, 0) corresponde a $x_1 = x_2 = x_3$ = falso, que satisface ϕ, tal como garantiza la equivalencia.

5.7 Conclusión de la reducción

PROMPT-DEC está en NP y 3-SAT se reduce a él en tiempo polinomial, así que PROMPT-DEC es NP-completo. En consecuencia, encontrar la configuración óptima con oráculo exacto es NP-difícil: no se conoce un algoritmo que lo resuelva en tiempo polinomial en el número de ranuras k.

Dos aclaraciones sobre el alcance:

- La dificultad se refiere al crecimiento con k. Con $k = 5$ y $m = 8$ fijos, el espacio es finito (32 768) y la fuerza bruta lo resolvería en tiempo constante pero impracticable (73 días de GPU). Lo que la NP-dureza indica es que ningún método general escapará del crecimiento exponencial cuando el catálogo crezca.
- Esta demostración es solo para la parte 1. La parte 2, con consultas costosas y ruidosas, se analiza por costo y por complejidad de muestreo (sección 3).

6. Proceso de optimización

La búsqueda encontró la configuración rol 3, estrategia 2, formato 0, estilo 3, cierre 3 (temperatura 0.0) con 124 evaluaciones registradas, el 0,38 % de las 32 768 posibles. Cada evaluación usa las mismas 100 instancias de búsqueda y la semilla 1; las repetidas salen del caché sin costo.

6.1 Justificación del método

Con 3,2 minutos por evaluación, la fuerza bruta (73 días) era inviable, y una búsqueda aleatoria desperdicia consultas porque la mayoría de las opciones dañan la respuesta. Se eligió un método en tres fases que aprovecha la estructura del problema: el prompt se divide en ranuras y las trazas indican qué familia de restricciones rompe cada opción (Tabla 6). No se usó recocido simulado ni algoritmos genéticos porque requieren cientos o miles de evaluaciones, lo que no cabía en el tiempo de GPU disponible.

Tabla 6

Fases del método de búsqueda

Fase	Celda	Qué hace	Parámetros	Parada	Evaluaciones
0 · Línea base	B	Mide el prompt sin instrucciones	—	—	1
1 · Cribado	B	Mide cada opción sola, con las demás ranuras vacías	5 ranuras × 8 opciones	Al recorrer las 40	40
Poda por trazas	C	Descarta opciones que bajan la precisión o rompen una familia	TOLERANCIA = 0,02; UMBRAL = 3; TOP = 3	—	0
2 · Ascenso sobre sobrevivientes	D	Cambia una ranura a la vez entre las opciones que sobrevivieron	máximo 3 pasadas	Pasada sin mejora estricta	13
3 · Ascenso sin poda	E	Cambia una ranura a la vez probando las 8 opciones en contexto	máximo 2 pasadas	Pasada sin mejora estricta	70

Un cambio solo se acepta si mejora estrictamente la precisión. Esta regla evita moverse entre configuraciones empatadas, que con 100 instancias no son distinguibles.

6.2 Fase 1: cribado

Sin instrucciones, el modelo acertó 16 de 100 instancias. De las 40 opciones medidas solas, solo 10 superaron esa línea base. En formato y cierre hubo una única opción que no empeoró: formato 0 (18 %) y cierre 3 (18 %). Las peores fueron formato 5 (6 %), cierre 7 (7 %) y formato 7 (7 %). La Tabla 7 resume el cribado por ranura.

Tabla 7

Resultado del cribado por ranura

Ranura	Mejor opción sola	Peor opción sola	Opciones sobre la base (16 %)
rol	2 y 3 (18 %)	4 (11 %)	2
estrategia	6 (19 %)	5 (10 %)	1
formato	0 (18 %)	5 (6 %)	1
estilo	0 (19 %)	4 (12 %)	4
cierre	3 (18 %)	7 (7 %)	1

6.3 Uso de las trazas para podar

Para cada opción se compararon sus trazas con las de la línea base, familia por familia. Una opción se descartó si su precisión quedó más de 2 puntos por debajo de la base o si aumentó en 3 o más los fallos de alguna familia. Por ejemplo, rol 1 (16 %, igual a la base) se descartó porque aumentó en 3 los fallos de `keywords:frequency`. De las sobrevivientes se conservaron las 3 mejores por ranura. El espacio pasó de 32 768 a 27 configuraciones: rol {2, 3, 0}, estrategia {6, 2, 3}, formato {0}, estilo {0, 7, 2} y cierre {3}.

6.4 Fase 2: interacción entre ranuras

La combinación de las mejores opciones individuales (rol 2, estrategia 6, formato 0, estilo 0, cierre 3) obtuvo solo 13 %, por debajo de la línea base, aunque cada opción por separado la superaba. Las ranuras interactúan: su efecto no se suma. Esta evidencia es coherente con la NP-dureza de la sección 5, que depende justamente de la interacción entre decisiones.

El ascenso sobre las sobrevivientes convergió en 17 % (rol 3, estrategia 2, formato 0, estilo 2, cierre 3) tras 2 pasadas. Como la poda se basó en efectos medidos de forma aislada, pudo descartar opciones que funcionan bien acompañadas. Por eso se agregó la fase 3.

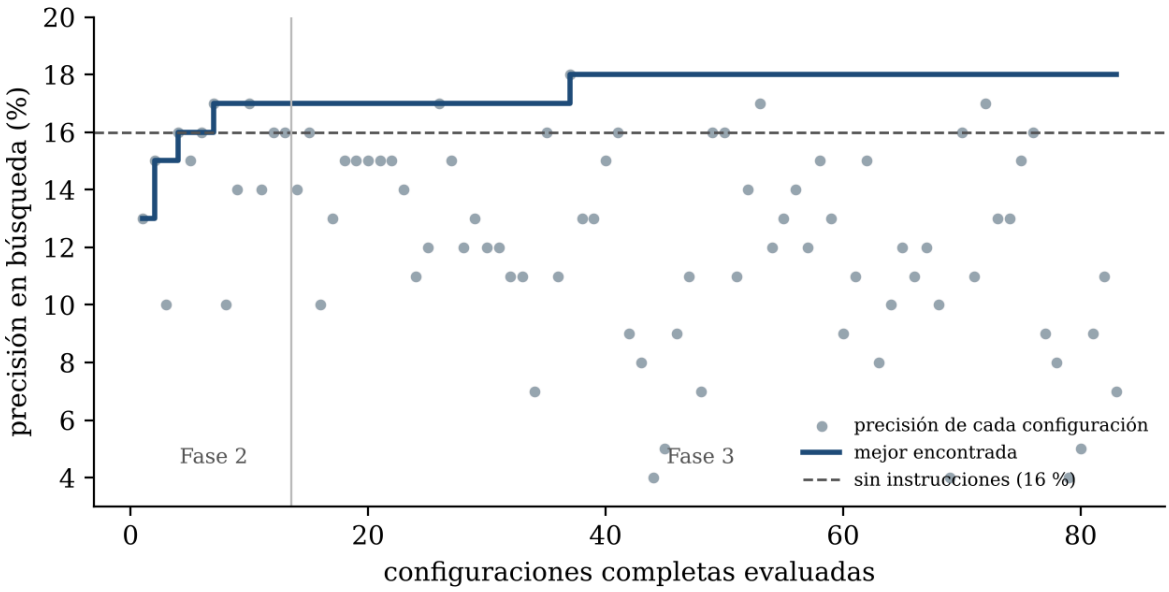
6.5 Fase 3: ascenso sin poda

Desde la configuración de 17 % se probaron las 8 opciones de cada ranura con las otras cuatro fijas. En la primera pasada, cambiar el estilo de 2 a 3 subió la precisión a 18 %. Estilo 3 había sobrevivido a la poda, pero quedó fuera del límite de 3 opciones por ranura: medido solo daba 16 %, y en contexto fue la mejor. La segunda pasada no encontró mejoras y el algoritmo se detuvo.

La misma interacción aparece en la fase 3: con estilo 3, cambiar la estrategia a 6 bajó la precisión a 11 %, aunque estrategia 6 era la mejor opción individual (19 %). En cambio, el cierre confirmó lo que mostró el cribado: con cualquier opción distinta de 3, la precisión cayó a entre 4 % y 11 %.

Figura 1

Mejor precisión encontrada en búsqueda a lo largo de las configuraciones completas evaluadas (fases 2 y 3)



Nota. Cada punto gris es una configuración con las cinco ranuras definidas; la línea azul es la mejor encontrada hasta ese momento. Elaboración propia a partir de `registro_busqueda.csv`.

6.6 Optimalidad del resultado

La configuración final es un óptimo local respecto a cambiar una ranura: ninguna de las 35 configuraciones vecinas evaluadas en la última pasada la supera. No es un óptimo global demostrado. Un óptimo mejor podría requerir cambiar dos o más ranuras a la vez, y el ascenso por coordenadas no explora esos movimientos.

En la búsqueda se observaron precisiones de hasta 19 % con una sola ranura activa (estrategia 6 o estilo 0 solos). Esas configuraciones no son soluciones válidas, porque `entrega.json` exige un índice en las cinco ranuras (sección 2.6). Además, la diferencia de 1 punto está dentro del error de medición (sección 3.3).

7. Resultados, validación y reproducibilidad

La configuración entregada acierta el 18 % de las instancias de búsqueda y el 21,8 % de las de validación, frente al 16 % y al 17,2 % sin instrucciones.

7.1 Configuración final

Tabla 8

Configuración final entregada en entrega.json

rol	estrategia	formato	estilo	cierre	temperatura
3	2	0	3	3	0.0

Tabla 9

Precisión de la configuración final frente a la línea base

Medida	Valor
Precisión en búsqueda (100 instancias)	18,0 % (18/100)
Precisión sin instrucciones en búsqueda	16,0 % (16/100)
Precisión en validación (87 instancias)	21,8 % (19/87)
Precisión sin instrucciones en validación	17,2 % (15/87)
Evaluaciones registradas	124 (como máximo 12 400 consultas al modelo)
Fracción del espacio evaluada	0,38 % de 32 768

7.2 Análisis de fallos: línea base frente a configuración final

La configuración final reduce de 84 a 82 las instancias fallidas en búsqueda. Las mejoras se concentran en familias de estructura y de conteo, y los nuevos fallos en familias de mayúsculas y de primera palabra (Tabla 10).

Tabla 10

Familias de restricciones con cambios de 2 o más fallos

Familia (primera restricción violada)	Base	Final	Cambio
`paragraphs:paragraphs`	4	0	−4
`keywords:word_once`	3	0	−3
`change_case:capital_word_frequency`	5	3	−2
`letters:letter_counting2`	2	0	−2
`change_case:english_capital`	3	5	+2
`change_case:english_lowercase`	0	2	+2
`detectable_format:sentence_hyphens`	2	4	+2
`first_word:first_word_answer`	2	4	+2

La tabla muestra solo las familias que cambiaron en 2 o más; la tabla completa está en la salida de la celda F del notebook de búsqueda. Las familias que empeoraron indican dónde buscar en una siguiente entrega: las de mayúsculas y minúsculas apuntan a la ranura de estilo, y las de primera palabra a las ranuras que van antes de la respuesta (rol y estrategia).

7.3 Validación

La llamada `oraculo.validar(mejor[1], n=None)` evaluó la configuración en las 87 instancias de validación, que no se usaron para elegirla. Obtuvo 21,8 %. Las restricciones más violadas fueron `punctuation:punctuation_dot` (8), `last_word:last_word_sent` (6) y `copy:repeat_phrase` (5).

La validación supera a la búsqueda en 3,8 puntos. Esa diferencia está dentro del error típico de medir con 87 instancias (unos 4,4 puntos con $p = 0{,}218$, sección 3.3), así que no indica una mejora real. Lo que sí indica es que la configuración no está sobreajustada a las 100 instancias de búsqueda. La comparación justa es con el prompt sin instrucciones sobre las mismas 87 instancias, que obtuvo 17,2 % (celda H). La configuración final acierta 4 instancias más (+4,6 puntos). La mejora aparece tanto en búsqueda (+2 puntos) como en validación, aunque con muestras de este tamaño no alcanza a ser estadísticamente concluyente.

Esta validación no es la evaluación final. La docente evaluará `entrega.json` sobre un conjunto de prueba privado al que el grupo no tiene acceso, que puede incluir familias de restricciones ausentes de los datos visibles.

7.4 Limitaciones

- La mejora sobre la línea base es pequeña (2 puntos en búsqueda) y del mismo orden que el error de medición con 100 instancias.
- El resultado es un óptimo local respecto a cambiar una ranura, no un óptimo global.
- Solo se exploró la temperatura 0.0 y un único modelo (`qwen17b`).
- Las trazas informan solo la primera restricción violada, así que el análisis de fallos no ve las demás.

7.5 Reproducibilidad

Todo está en el repositorio Manuel29296/ADA-2026-2-GP01, rama `main` (Tabla 11).

Tabla 11

Archivos del repositorio que respaldan el resultado

Archivo	Contenido
`entrega.json`	La configuración final
`busqueda_G01.ipynb`	Notebook de búsqueda con sus salidas: línea base, cribado, poda, ascenso, análisis de fallos y validación
`registro_busqueda.csv`	Las 124 evaluaciones con su precisión y las familias más violadas
`evaluacion_G01.ipynb`	Notebook de evaluación para la docente, sin parámetros que editar
`docs/MANUAL_TECNICO.md`	Manual técnico

Se usaron el modelo `Qwen/Qwen3-1.7B`, temperatura 0.0, semilla 1 y las particiones de `dividir`. Con temperatura 0.0 la generación es determinista, así que repetir el notebook de búsqueda reproduce las mismas precisiones.

Referencias

Cook, S. A. (1971). The complexity of theorem-proving procedures. En *Proceedings of the Third Annual ACM Symposium on Theory of Computing* (pp. 151–158). ACM. https://doi.org/10.1145/800157.805047

Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. W. H. Freeman.

Hoeffding, W. (1963). Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association, 58*(301), 13–30. https://doi.org/10.1080/01621459.1963.10500830

JuManoel. (2026). *ADA-2026-2-GP01* [Repositorio de software]. GitHub. https://github.com/JuManoel/ADA-2026-2-GP01

Karp, R. M. (1972). Reducibility among combinatorial problems. En R. E. Miller & J. W. Thatcher (Eds.), *Complexity of computer computations* (pp. 85–103). Plenum Press.

Universidad de Caldas. (2026). *Primera entrega: qué debe presentar cada grupo. Guía de productos y criterios mínimos de entrega · ADAG1* [Documento del curso Análisis y Diseño de Algoritmos].